Software Design Process

Introduction

- Software design is a critical phase in the software development lifecycle.
- This process converts requirements into a comprehensive design for the final product, providing essential details regarding:
    1. Data structures
    2. Software architecture
    3. Interfaces
    4. Components
- These procedural details are fundamental for the successful implementation of the software system.

Software Design Concepts

Key concepts in software design include:

- Abstraction
- Modularity
- Refactoring
- Functional Independence
- Cohesion
- Coupling

---

1. Abstraction

- Abstraction is the procedural removal of non-essential details to highlight only critical features.
- It simplifies complexity, ensuring that sophisticated systems are clear to understand and manage.

Example: An ATM interface presents Withdrawal and Deposit options while isolating internal processing.

---

2. Modularity

- Modularity involves the division of a system into distinct, independent modules.
- Individual modules facilitate isolated development, testing, and maintenance.

It optimizes:

- Flexibility
- Reusability
- Scalability

Example: Library system modules such as Login, Book, and Report modules.

---

3. Refactoring

- Refactoring improves the internal code structure while preserving existing external behavior.

This process assists in:

- Improving code readability
- Removing redundant code
- Enhancing maintainability

Example: The simplification of logic and renaming of variables.

---

4. Functional Independence

- Functional independence is achieved when a module executes a single, clear task with minimal cross-module interaction.
- This is a fundamental characteristic of high-quality software design that ensures reliability and modularity.

Example: A Payment module exclusively managing financial operations.

---

5. Cohesion

1. Cohesion measures the strength of relationships between functions within a single module.
2. High cohesion produces focused modules that support efficient information hiding and maintenance.

Example: A Student module limited strictly to handling student records.

---

6. Coupling

- Coupling evaluates the degree of dependency existing between separate modules.
- Low coupling indicates that modules are independent and easy to modify without impacting the system.

Example: A Login module functioning independently of the Report module.

Difference Between Cohesion and Coupling

Cohesion	Coupling
Represents relationships within a module.	Represents relationships between modules.
High cohesion is prioritized.	Low coupling is prioritized.
The resulting module is highly focused.	Resultant modules are independent.

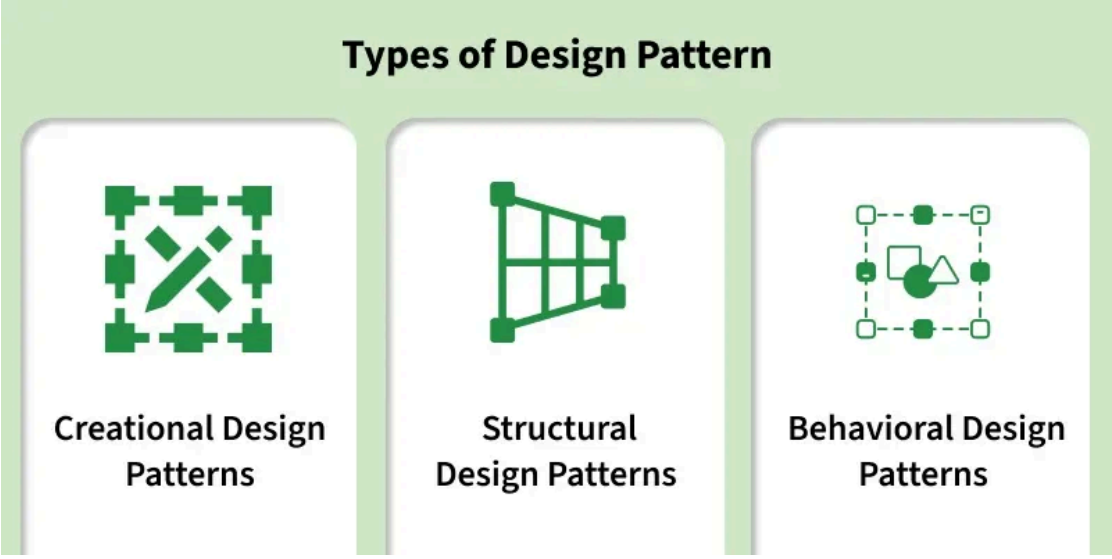
Design Pattern

- A Design Pattern represents a reusable, common solution for recurring software design challenges.
- These patterns facilitate more efficient and rapid development for engineering teams.

Types of Design Patterns

The three primary classifications are:

- Creational Patterns
- Structural Patterns
- Behavioural Patterns



1. Creational Patterns

- These patterns govern object creation processes, ensuring flexibility and control over instantiation.

Examples:

- Singleton, Factory Method, Abstract Factory, Builder, and Prototype.

2. Structural Patterns

- Structural patterns address the composition of classes and objects into larger, more complex systems.

Examples:

- Adapter, Bridge, Composite, Decorator, Facade, and Proxy.

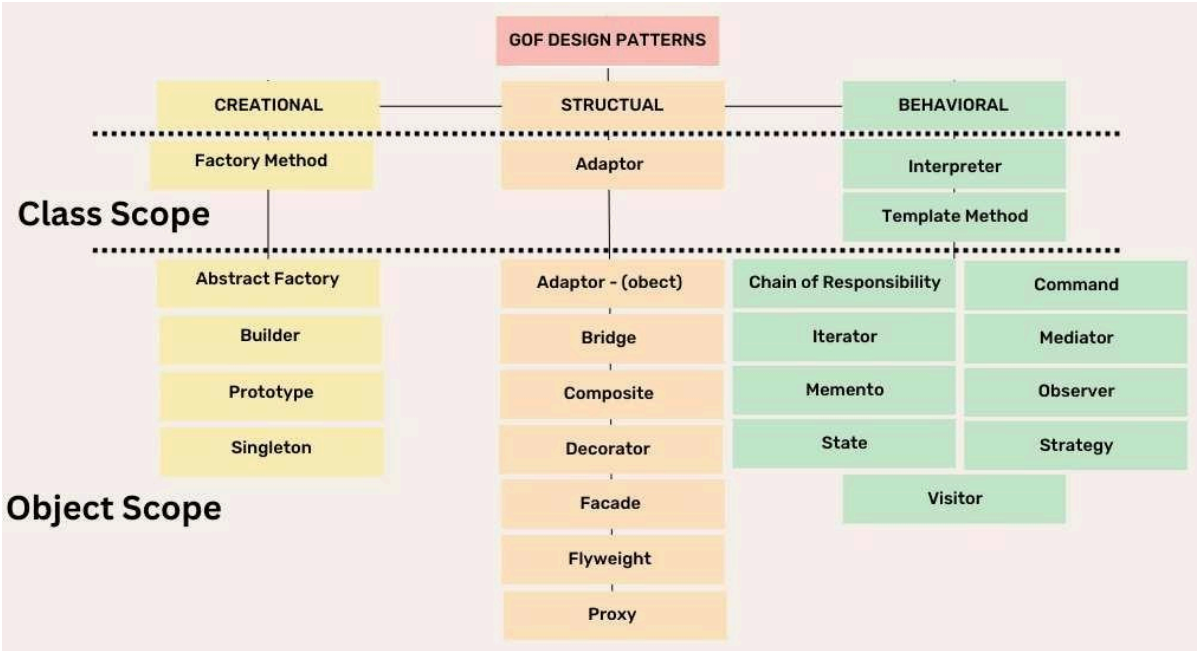
---

3. Behavioural Patterns

- These patterns facilitate effective communication and the assignment of responsibility among objects.

Examples:

Observer, Strategy, Command, Iterator, Mediator, and State.



Advantages of Design Patterns

Key benefits include:

- Provision of reusable solutions for common design issues.
- Significant reduction in development lifecycle time.
- Simplification of system maintenance and modifications.
- Enhanced coordination and communication among developers.
- Overall improvement in software quality and robustness.

Coupling, Cohesion and Functional Independence

1. Coupling

Coupling evaluates the degree of dependency or interconnection existing between various software modules.

Within high-quality software design, **low coupling is prioritized** to facilitate ease of modification, testing, and maintenance.

Types of Coupling

1. **Content Coupling:**
   One module directly modifies or relies on the internal workings of another (Very Bad)
2. **Common Coupling:**
   Modules share global data, leading to unpredictable changes
3. **Control Coupling:**
   One module controls the logic flow of another by passing a "control flag"
4. **Stamp Coupling:**
   Modules share a complex data structure but only use specific parts of it
5. **Data Coupling:**
   Modules only share necessary data, which is better for independence
6. **Message Coupling:**
   modules interact only via message passing or interfaces (The Best)

Order of Coupling

Content → Common → Control → Stamp → Data → Message

2. Cohesion

Cohesion evaluates the strength of internal relationships between elements dedicated to a single task within a module.

In robust architectural design, **high cohesion is prioritized**.

Types of Cohesion

1. **Coincidental Cohesion:**
   Elements within a module are grouped together randomly with no meaningful relationship
2. **Logical Cohesion:**
   Elements perform similar activities but are technically unrelated
3. **Temporal Cohesion:**
   Elements are related only because they are executed at the same time (e.g., initialization)
4. **Procedural Cohesion:**
   Elements follow a specific sequence of execution to achieve a task
5. **Communication Cohesion:**
   All elements operate on the same set of input or output data
6. **Functional Cohesion:**
   Every element in the module contributes to a single, well-defined task (The Best)

Order of Cohesion

Coincidental → Logical → Temporal → Procedural → Communication → Functional

---

3. Functional Independence

Functional Independence is achieved when modules are characterized by:

- High Cohesion
- Low Coupling

Every module must be designed to execute a single task with minimal external dependency.

Significance of Functional Independence

- **Ease of Maintenance:** Changes in one module won't break others
- **Improved Testing:** Modules can be tested in isolation
- **Reduced Complexity:** Simplifies the overall system design and enhances team collaboration

---

Difference Between Coupling and Cohesion

Coupling	Cohesion
Represents relationships existing between separate modules.	Represents relationships existing within a single module.
The prioritization of low coupling is essential.	The prioritization of high cohesion is essential.
Evaluates the degree of module dependency.	Evaluates the functional focus of the module.

Architectural Design Model

The architectural design model functions as a structural blueprint for the system, prioritizing fundamental design decisions at an early stage of the software development lifecycle.

Architectural Design Styles describe:

- A collection of hardware and software components responsible for executing required functions *(e.g., Databases, Modules, Frameworks).*
- A set of connectors facilitating coordination and communication between various components.
- Specific conditions for the successful integration of components into a unified system.
- Semantic models utilized to comprehend the overall properties of the system.

Importance of Software Architecture

1. **Security:** Safeguards the system through the implementation of encryption and various security mechanisms.
2. **Performance:** Optimizes the system to handle requests and responses with high efficiency.
3. **Maintainability:** Ensures the ease of modifying or replacing individual software components.
4. **Safety:** Minimizes operational risk by isolating critical system functionalities.
5. **Availability:** Effectively manages errors to guarantee continuous system operation.

Architectural Views

- Architectural views are utilized to represent the entire system architecture in a manner that is meaningful to stakeholders.

Types of Views:

1. **Use Case View:** Represents the user perspective based on requirements using use case diagrams.
2. **Design View:** Illustrates the design structure, key features, entities, and their attributes.
3. **Process View:** Describes system communication, behavior, and synchronization processes.
4. **Implementation View:** Focuses on source code organization, developer roles, and system integration.
5. **Deployment View:** Displays the physical system environment where the application resides and executes.

Taxonomy of Architectural Styles

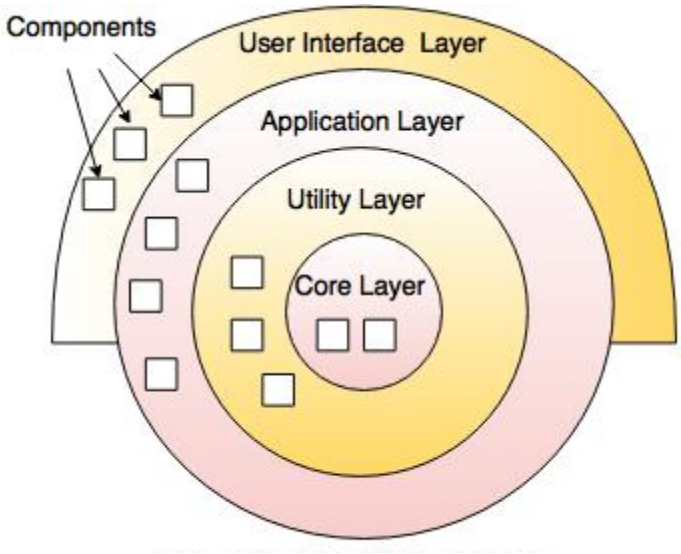
Architectural styles establish the fundamental structure and organization of system components.

Types:

- Data Centered Architecture
- Data Flow Architecture
- Object Oriented Architecture
- Layered Architecture

Layered Architecture

- In this model, data transitions through multiple layers for sequential processing.
- Every layer is designed to perform a specific and well-defined function.



Layers:

- **Outer Layer:** Manages all user interface operations.
- **Intermediate Layers:** Provide utility functions and application-level services.
- **Inner Layer:** Facilitates direct interaction with the Operating System.

Object Oriented Architecture

- Objects serve as the fundamental building blocks of the software system.

Concepts:

Object: An instance of a class *(e.g., Student)*.

Class: Defines the specific attributes and methods for objects.

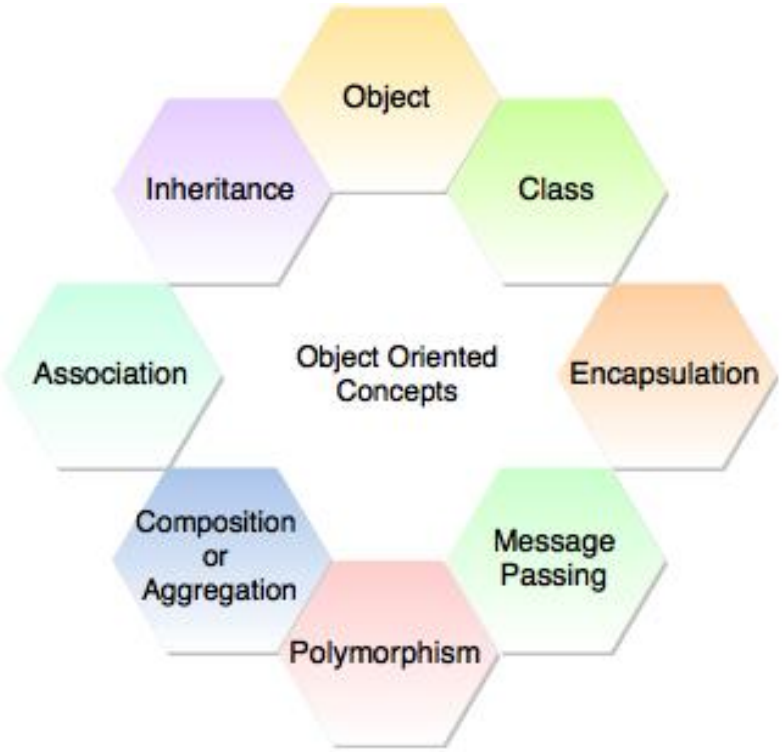
Encapsulation: The process of binding related elements together.

Abstraction: The removal of non-essential details to simplify complexity.

Inheritance: Creating a new class derived from an existing one.

Polymorphism: Allows a single object to take on multiple forms.

Message Passing: Enables coordination and communication between objects.



Data Flow Architecture

- This architecture transforms input data into output results through a sequence of processing components.

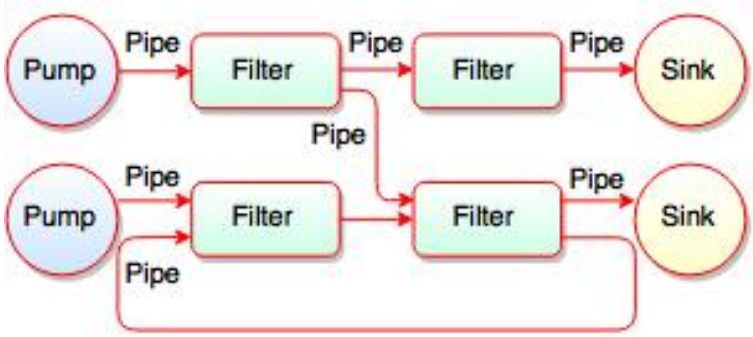


Fig. Pipes and Filters

Components:

- **Pipe:** Acts as a conduit for transferring data between system components.
- **Filter:** Processes incoming data streams to produce meaningful output.

Advantages:

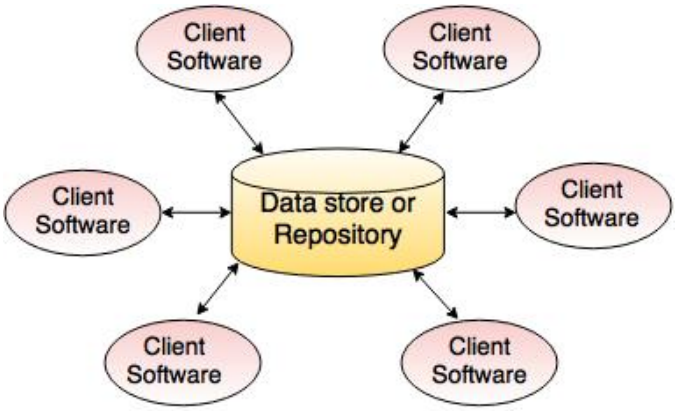
- Supports the concurrent execution of various processes.

Disadvantages:

- Typically provides less opportunity for direct user interaction.

Data Centered Architecture

- Data is maintained in a centralized location for efficient storage and management.
- The architecture allows multiple users to access the data simultaneously.



Features:

- Enables users to add, update, delete, and modify stored data records.

Advantages:

- Operates independently of individual client configurations.
- Facilitates the seamless addition of new clients to the system.
- Simplifies the modification of system components